

Análisis de Código OpenMP y Comportamiento de Hilos

1. Explicación del Código Línea por Línea

Inclusiones y Configuración Inicial

- `#include <stdio.h>` y `#include <stdlib.h>`: Librerías estándar de C para entrada/salida y gestión de memoria dinámica (`malloc`, `free`, `rand`).
- `#include <omp.h>`: Librería fundamental de OpenMP que provee las directivas del compilador y funciones de la API (como `omp_get_thread_num`).
- `#include <time.h>`: Utilizada para inicializar la semilla de los números aleatorios basándose en el reloj del sistema.

Función Principal y Semilla

- `int main() { ... }`: Punto de entrada del programa.
- `srand(time(NULL));`: Inicializa el generador de números aleatorios para que en cada ejecución del programa los valores del vector sean distintos.
- `omp_set_num_threads(3);`: Fuerza al entorno de ejecución de OpenMP a utilizar exactamente 3 hilos para las siguientes regiones paralelas.
- `int tamaños[] = {10000, 100000, 500000};`: Define un arreglo con los tres tamaños de vector solicitados.
- El bucle `for` itera sobre este arreglo, llamando a la función `procesar_vector(N)` para cada tamaño.

Inicialización del Vector (`procesar_vector`)

- `double *vec = (double *)malloc(N * sizeof(double));`: Asigna memoria dinámica en el montón (heap) contigua para albergar `N` elementos de tipo `double`.
- El bucle `for (int i = 0; i < N; i++) { vec[i] = (double)(rand() % 100); }`: Rellena el vector con números pseudoaleatorios entre 0 y 99.
- Se declaran `min_val`, `max_val`, `suma` y `promedio`, inicializando los dos primeros con el primer elemento del vector para tener un punto de comparación válido.

Región Paralela y Distribución del Trabajo

- `#pragma omp parallel`: Directiva que crea un equipo de hilos (en este caso, 3 hilos según se definió en `main`). A partir de esta línea, el código encerrado entre llaves es ejecutado por todos los hilos de forma concurrente.
- `#pragma omp master`: Asegura que el bloque de código siguiente sea ejecutado **exclusivamente por el hilo maestro** (el hilo con ID 0). Los demás hilos ignoran este bloque.
 - `printf("... %d ... %d ", omp_get_thread_num(), omp_get_num_threads());`: El hilo maestro imprime su ID (siempre 0) y el número total de hilos en el equipo actual (3).
- `#pragma omp sections`: Directiva de reparto de trabajo (worksharing). Indica que las subsecciones internas deben distribuirse entre los hilos del equipo. No todos los hilos ejecutan todas las secciones; cada sección es ejecutada por un único hilo.
- `#pragma omp section` (Repetida 3 veces): Delimita cada bloque de trabajo independiente.
 - **Sección 1 (Máximo)**: Un hilo toma esta sección, obtiene su ID con `omp_get_thread_num()` e itera sobre todo el vector buscando el valor más grande. Utiliza una variable local `max_local` para evitar problemas de concurrencia y falsos compartimientos de memoria, volcando el resultado a `max_val` al final.
 - **Sección 2 (Mínimo)**: Otro hilo procesa esta sección de manera análoga a la anterior, buscando el valor más pequeño y guardándolo en `min_val`.
 - **Sección 3 (Promedio)**: El hilo restante itera sobre el vector sumando todos sus elementos en `suma_local`, volcando el resultado final en `suma`.
- Al finalizar el bloque `#pragma omp sections`, existe una **barrera implícita**. Ningún hilo continuará más allá de este punto hasta que las tres secciones hayan terminado su ejecución.

Finalización Secuencial

- Una vez que todos los hilos terminan y la región paralela se cierra, el hilo maestro (ejecutando en solitario) calcula el promedio final: `promedio = suma / N;`.
- Se imprimen los resultados calculados y se libera la memoria del vector con `free(vec);` para evitar fugas de memoria.

2. Interpretación de los Resultados

Al analizar la salida de ejecución, se extraen las siguientes conclusiones clave sobre el comportamiento concurrente de OpenMP y la correctitud del algoritmo:

Comportamiento Estadístico (Correctitud)

Para los tres tamaños (N=10000, 100000 y 500000), los resultados convergen lógicamente:

- **Mínimo y Máximo:** El mínimo siempre es `0.00` y el máximo siempre es `99.00`. Dado que los valores generados están limitados entre 0 y 99 (`rand() % 100`), un vector a partir de 10,000 elementos es estadísticamente lo suficientemente grande como para garantizar la aparición de los extremos absolutos en todos los casos.
- **Promedio:** Los valores oscilan muy cerca del `49.5` (49.52, 49.48, 49.41). Esto es matemáticamente correcto, ya que la media esperada de una distribución uniforme de números enteros entre 0 y 99 es exactamente 49.5.

Mapeo de Hilos (Worksharing)

El entorno OpenMP asignó las secciones de la siguiente manera durante toda la ejecución:

- La **Sección 3 (Promedio)** fue asignada sistemáticamente al **Hilo 0** (el hilo maestro).
- La **Sección 1 (Máximo)** fue asignada al **Hilo 1**.
- La **Sección 2 (Mínimo)** fue asignada al **Hilo 2**.

Esto demuestra que `#pragma omp sections` distribuye exitosamente las tareas. Aunque el algoritmo de asignación es manejado internamente por el compilador (y no se garantiza que siempre sea el mismo orden en ejecuciones futuras), en este caso resolvió que cada uno de los 3 hilos procesara exactamente una de las 3 secciones.

Concurrencia y Condición de Carrera en la Salida (I/O)

La evidencia visual más importante de la ejecución paralela en la consola es el **no determinismo** en el orden de las impresiones en pantalla:

- Para **N = 10000**, el hilo maestro imprime el encabezado "Cantidad total de hilos..." primero, seguido por la información de los hilos 0, 1 y 2.
- Sin embargo, para **N = 100000** y **N = 500000**, el hilo ID 1 (encargado de la Sección 1 - Máximo) imprime su línea de procesamiento **antes** de que el Hilo Maestro alcance a imprimir la cantidad total de hilos.

Esto ocurre porque todos los hilos se lanzan de manera simultánea al inicio de la región paralela.

La directiva `#pragma omp master` no frena a los demás hilos; estos continúan hacia las secciones. El Hilo 1 simplemente llegó a su instrucción `printf` microsegundos más rápido que el Hilo 0 en esas iteraciones específicas. Esto ilustra que, sin barreras explícitas de sincronización, no se puede predecir ni garantizar el orden cronológico en el que los hilos concurrentes accederán a recursos compartidos como la salida estándar (`stdout`).